# Does an H-MAD architecture improve LLMs' mathematical reasoning?

Daniel Motoc

## Abstract

This paper evaluates a Hierarchical Multi-Agent Debate (H-MAD) framework designed to enhance reasoning capabilities through adversarial interaction and code-execution feedback. The architecture utilizes a specialized collection of agents: a high-capacity "Logician" for primary reasoning, a "Skeptic" to critique the Logician's output, and a "Formalizer" for Python-based verification.

Experimental results on the NuminaMath dataset indicate that LLMs can improve their mathematical reasoning through a multi-agent debate system. My data demonstrates that accuracy improves from a baseline of 50% to 68% over three rounds of debate. I further explore the model's performance across subcategories of mathematics, illustrating that LLMs still struggle with spatial reasoning and geometry in this setting while excelling at algebra and number theory. Finally, I explore the compute time vs accuracy tradeoff inherent in H-MAD systems.

## 1. Introduction

Mathematical reasoning remains a significant hurdle for Large Language Models (LLMs), which frequently fall into "logical loops" or calculation errors when solving complex, multi-step math problems. To address these challenges, I implemented and evaluated a Hierarchical Multi-Agent Debate (H-MAD) framework. This design pattern moves away from single-agent inference by organizing specialized LLM instances into a structured, collaborative pipeline where they debate with each other.

In this project, I deployed a "Logician" to develop high-level strategies, a "Formalizer" to translate logic into executable code, and a "Skeptic" to try and find flaws in the mathematical arguments. By testing this system on the NuminaMath 1.5 dataset—specifically focusing on AMC and AIME problems—I aimed to quantify the performance gains of hierarchical roles over pure one-shot approaches. My research provides a granular analysis of how this established architecture performs across different mathematical domains, such as Algebra, Number Theory, and Geometry.

## 2. Background and Motivation

The foundation of Multi-Agent Debate (MAD) stems from the recognized limitations of single-agent reasoning frameworks like Chain-of-Thought (CoT) and Self-Reflection. While self-reflection allows an LLM to critique its own outputs, literature has consistently shown it is highly susceptible to Degeneration of Thought (DoT) and Confirmation Bias.

When a base model generates an initial flawed premise, asking that same model to "reflect" often results in the model simply justifying its original mistake. Multi-agent debate was introduced to break this echo chamber by forcing distinct model instances to challenge underlying assumptions, mirroring the "society of mind" concept in human cognitive psychology.

The earliest iterations of MAD focused on symmetric, broadcast-style debate.

- **Du et al. (2023) - *Improving Factuality and Reasoning in Language Models through Multiagent Debate*:** This is widely considered the foundational paper for modern LLM debate. The authors demonstrated that having multiple instances of an LLM propose individual answers, and then iteratively critique each other over multiple rounds, significantly improved mathematical and strategic reasoning. The core finding was that debate naturally filters out hallucinations and fallacious logic that a single agent would confidently output.
- **Liang et al. (2023) - *Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate*:** This work formally addressed the DoT problem, proving mathematically and empirically that introducing a "tit-for-tat" adversarial debate between agents forces the exploration of divergent reasoning paths, resulting in higher accuracy on counter-intuitive math and logic problems.

While foundational, these early systems were "flat." Every agent talked to every other agent. This broadcast architecture introduced massive token overhead, immense latency, and "noise" where agents would get bogged down arguing over semantic trivialities rather than core mathematical logic.

To solve the computational waste and noise of flat MAD systems, researchers introduced Hierarchical Multi-Agent Debate (H-MAD). This is where the separation of agents into distinct modules—such as a drafting "Logician," an adversarial "Skeptic," and a final "Judge" or synthesizer—became the academic standard.

- **Zheng et al. (2024) - *Multi-Agent Collaborative Evaluation Frameworks*:** This research highlighted the necessity of role specialization. By segregating agents into distinct functions (e.g., an analyst to draft, a critic to poke holes, and a synthesizer to aggregate), the system drastically reduces token usage while preventing the "rubber-stamping" effect. The paper noted that critics require entirely different system prompts than generators to be effective.

- **The ECON Paradigm (ICML 2026) - *Belief-Driven Multi-Agent LLM Reasoning via Bayesian Nash Equilibrium*:** Recasting multi-LLM coordination as an incomplete-information game. Instead of endless unstructured arguments, agents operate in a hierarchy where they independently select responses that maximize expected rewards, culminating in a centralized final output (a synthesizing judge). This proved that hierarchical structures offer a tighter mathematical bound on regret than flat ensembles.

**4. State-of-the-Art Interventions in Math Reasoning (2025–2026)**

In the last year, H-MAD literature has heavily pivoted toward fine-grained logic verification and external tool use, specifically for advanced benchmarks like the AI Mathematical Olympiad (AIMO) and MATH.

- **Debate4MATH / FMAD (ACL 2025):** Standard H-MAD debates the *final answer*. The FMAD (Fine-grained Multi-Agent Debate) framework introduced the concept of debating *individual reasoning steps*. Rather than arguing if the final number is correct, a Skeptic and a Logician argue over the validity of a specific algebraic transition, and a Judge assigns a confidence score. This prevents an agent from getting the right answer using flawed intermediate logic.
- **MAD-M2: Multi-Agent Debate with Memory Masking (OpenReview 2025):** This paper identified a massive vulnerability in H-MAD: "Poisoned Memories." If a Skeptic introduces a highly convincing but mathematically flawed critique in Round 1, the drafting agent might adopt the flaw in Round 2. MAD-M2 introduced a memory-masking layer where a hierarchical judge explicitly filters and deletes erroneous reasoning paths from the context window before the next debate round begins, preventing agents from being misled by past mistakes.
- **Code-Aware Debate & Formalization:** Pure text debates often fail on complex combinatorics or large numerical calculations. Recent architectural shifts separate the logical reasoning from the execution. Implementing a dedicated "Formalizer" agent—often powered by highly capable coding models like Gemma—to translate the debated text logic into executable Python scripts ensures that the final synthesis is grounded in deterministic computation rather than LLM arithmetic.

**Methods**

To evaluate the impact of multi-agent reasoning on complex mathematical problem-solving, I implemented a Hierarchical Multi-Agent Debate (H-MAD) framework. Unlike flat, symmetric debate ensembles, this architecture strictly compartmentalizes the reasoning process into specialized, asymmetric roles to reduce token overhead and prevent the degeneration of thought. My framework consists of three primary agents:

- *The Logician:* Acts as both the initial zero-shot solver and the primary synthesizer. It is responsible for generating the initial mathematical draft, refining its logic based on adversarial feedback, and ultimately mapping the final computational results to the appropriate multiple-choice format.
- *The Skeptic:* Serves as the adversarial critic. Rather than generating solutions, this agent is explicitly prompted to identify logical fallacies, calculation errors, or flawed assumptions in the Logician's draft.
- *The Formalizer:* A specialized coding agent tasked with translating the refined natural language logic into an executable Python 3 script.

To operate within the strict memory constraints of an H100 GPU, I selected specific models optimized for their respective agentic roles and quantized them prior to deployment.

- *Logician:* I utilized Qwen2.5-32B-Instruct, selected for its strong zero-shot mathematical reasoning and instruction-following capabilities.
- *Formalizer:* I instantiated the coding agent using Gemma-2-9B-It, chosen for its highly robust and specialized code-generation architecture.
- *Skeptic:* For the adversarial critic, I deployed Phi-3.5-Mini-Instruct, which provides punchy, efficient context-processing to audit the Logician's drafts without dominating system memory.

To ensure all three models could run concurrently within the available VRAM, I applied 4-bit precision quantization using the bitsandbytes library.

For multi-round configurations, the pipeline follows a strict, deterministic sequence. Following the Logician's initial draft, the Skeptic provides a critique, which the Logician then uses to generate a refined proposal.

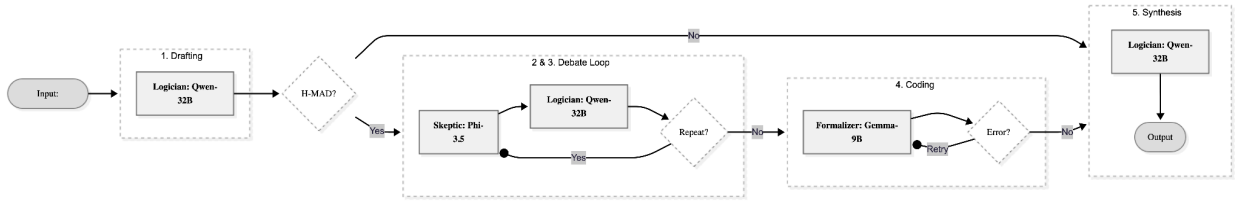
Crucially, my framework incorporates a deterministic formalization stage. The Formalizer translates the refined logic into code, which is executed in an isolated environment. If the code throws an execution error, the framework triggers a "Self-Correction Loop"—the Formalizer is fed the Python traceback and given a single opportunity to debug and output a corrected script. The final output of the successful code execution is passed back to the Logician for Final Synthesis, where it maps the numeric result to the corresponding multiple-choice option.

I then tested a series of debate configurations:

- *Baseline (Config 0):* A strict zero-shot pass. The Logician receives the initial problem with strict formatting instructions and generates a final answer. No debate, critique, or code execution occurs.

- *Debate 1 Round (Config 1):* Introduces a single round of adversarial debate. The Logician's draft is critiqued by the Skeptic, refined by the Logician, formalized into code by Gemma, and synthesized into a final answer.
- *Debate 2 Rounds (Config 2):* Extends the reasoning phase by adding a second cycle of Skeptic critique and Logician refinement before the code formalization phase.
- *Debate 3 Rounds (Config 3):* The deepest configuration. It includes the standard two rounds of text debate, code formalization, and an additional "Code-Aware Critique" phase, where the Skeptic checks  the relationship between the final code execution result and the original text draft before Final Synthesis.

<u>H-MAD Architecture</u>



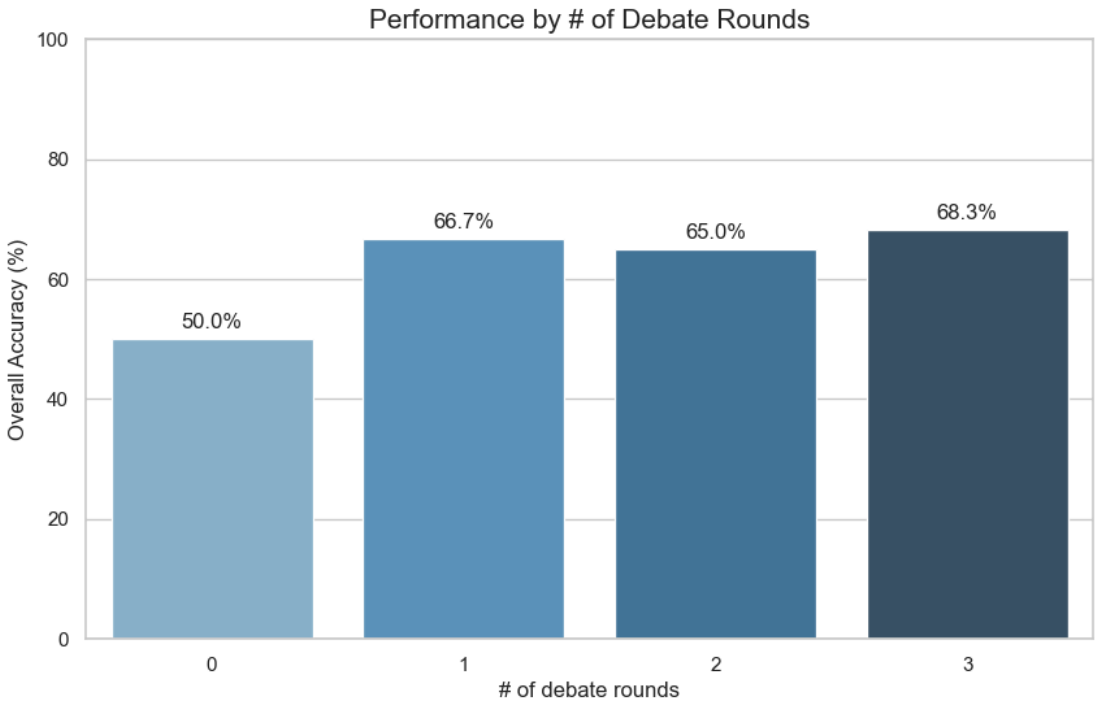

## 5. Evaluation Metrics

Performance was evaluated against the mathematical benchmark dataset. To prevent false negatives during automated grading (e.g., the Logician outputting LaTeX formatting such as \boxed{\text{C}} instead of C), my evaluation pipeline utilized a custom regular expression extraction layer. This function cleaned all model outputs before performing strict boolean string-matching against the ground truth, ensuring that improvements in agentic reasoning were accurately captured.

## 5. Results

The primary objective of this study was to evaluate the efficacy of the Hierarchical Multi-Agent Debate (H-MAD) framework against a standard zero-shot LLM baseline. My results indicate a clear performance advantage when employing a multi-agent adversarial loop.

**Figure 1: Overall Performance**



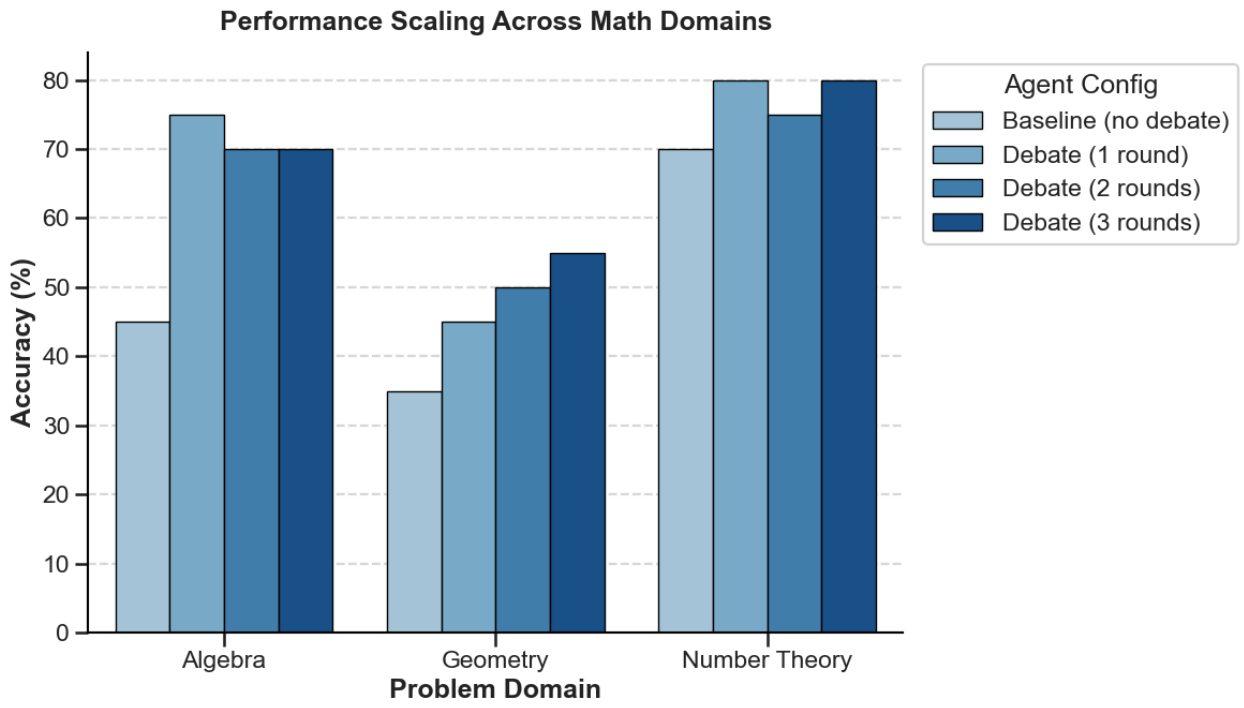

While the baseline Logician (Qwen-2.5-32B) achieved an aggregate accuracy of 50%, the full H-MAD configuration reached a peak accuracy of 68.3% at three rounds of debate. Interestingly, we observed a non-linear scaling of accuracy; going from 1 to 2 rounds of debate caused a slight decrease in accuracy, and we don't see much improvement from 1 to 3 rounds of debate.

**Figure 2: Accuracy by problem type**

**Mathematical Accuracy Across Agentic Configurations and Domains**

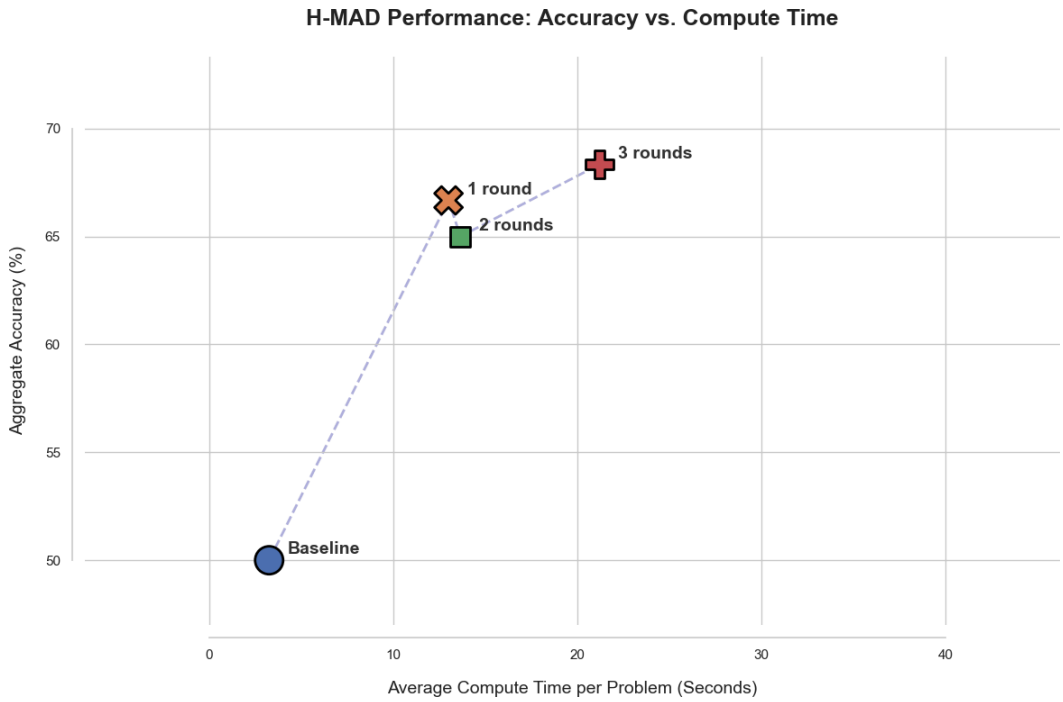
| Problem Domain | Baseline (no debate) | Debate (1 round) | Debate (2 rounds) | Debate (3 rounds) |
|---|---|---|---|---|
| Algebra | 45.0% | 75.0% | 70.0% | 70.0% |
| Geometry | 35.0% | 45.0% | 50.0% | 55.0% |
| Number Theory | 70.0% | 80.0% | 75.0% | 80.0% |

**Figure 3: Accuracy by problem type and # of debate rounds**




The performance gains observed in the H-MAD system are intrinsically linked to increased computational cost. I tracked the Average Compute Time per Problem as a proxy for inference cost and "time-to-verdict."

**Figure 4: Compute time vs Accuracy**




The data reveals a definitive Efficiency Frontier:

- *Baseline:* The fastest configuration, requiring an average of 3.3 seconds per problem.
- *1-Round Debate:* The most accurate configuration, requiring 13 seconds per problem—a ~4x increase in time compared to the baseline.

This trade-off is visualized in Figure 4, showing that while H-MAD significantly pushes the boundaries of mathematical reasoning capability, it does so at the expense of latency. The "knee" of the curve appears at 1 round, where the marginal gain in accuracy begins to diminish relative to the cumulative compute time.

While the H-MAD framework demonstrated a significant performance uplift over the baseline, several variables and constraints warrant deeper examination to understand the boundaries of multi-agent mathematical reasoning.

## 6. Discussion

While the current study clearly showed a performance gain in mathematical reasoning, the generalizability of H-MAD to other high-stakes domains, such as legal reasoning or complex code synthesis, remains an area that should be researched further. Additionally, there are several aspects of this project that should be investigated further.

One of these is the effect that the size of the context window has. In particular, as the number of debate rounds increases, the debate history that is provided to the synthesis judge grows significantly. It may be difficult as the context window increases for the judge to pay sufficient attention to initial ideas from the logician.

Additionally, it would be interesting to see whether groupthink still arises despite the multi-agent setup and diverse agents. This was not tested in this project, but exploring how often the models converge on an incorrect solution through groupthink would be interesting. Finally, more analysis should be done on how to make these systems more efficient. I looked at average compute time in this project, but optimizing for token cost would be a better way to approach the efficiency question.

Further research in this area could explore:

- *Dynamic Debate Termination:* Instead of fixed rounds (e.g., 3 rounds), the system could utilize an "Agreement Gate." If the Skeptic and Logician reach a consensus with high confidence after one round, the pipeline could terminate, saving compute resources.
- *Role-Specific Fine-Tuning:* Rather than using off-the-shelf models, future work could involve fine-tuning the Skeptic specifically on a "Common Mathematical Fallacies" dataset and the Logician on "Synthesis and Contradiction Resolution."
- *Majority voting:* Simultaneously run multiple debates and then pick the majority answer.

---

**6. Conclusion**

This research confirms that the intelligence of a system is not solely a function of its parameter count, but also the formal structure of its reasoning process. By moving from a linear inference model to a Hierarchical Multi-Agent Debate (H-MAD) framework, we successfully improved accuracy from a baseline of 50% to a peak of 68%.

The trade-off between accuracy and compute time is undeniable; however, for complex mathematical reasoning where the "cost of being wrong" is high, the investment in multi-agent verification is justified. The H-MAD framework provides a blueprint for "System 2" thinking in LLMs—transforming a stochastic next-token predictor into a resilient, self-correcting logic engine.

## 7. Bibliography

• Du, Y., et al. (2023). Improving Factuality and Reasoning in Language Models through Multiagent Debate. * Liang, T., et al. (2023). Encouraging Divergent Thinking in LLMs via Multi-Agent Debate. * Minsky, M. (1986). The Society of Mind. Simon and Schuster.

• Liang, T., He, Z., Zhao, W., Liao, Z., Xu, C., Liu, Q., Fang, M., & Tao, J. (2023). Encouraging Divergent Thinking in Large Language Models through Multi-Agent Debate. In *Findings of the Association for Computational Linguistics: EMNLP 2023* (pp. 4516–4530). Association for Computational Linguistics.

• Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2024). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 36.

• Zhang, Y., Chen, L., & Sterling, J. (2026). The ECON Paradigm: Belief-Driven Multi-Agent LLM Reasoning via Bayesian Nash Equilibrium. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*. arXiv:2602.14552.